

Domanda 18 Dare la definizione di $\Theta(f(n))$. Mostrare che $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$.

$$\Theta(f(n)) = \{g(n) \mid \exists n_0 \in \mathbb{N}. \exists c, c_2 > 0. \forall n \geq n_0. 0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)\}.$$

Per dimostrare che $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$, ricordiamo che

1. $O(f(n)) = \{g(n) \mid \exists n_0 \in \mathbb{N}. \exists c > 0. \forall n \geq n_0. 0 \leq g(n) \leq cf(n)\}$
2. $\Omega(f(n)) = \{g(n) \mid \exists n_0 \in \mathbb{N}. \exists c > 0. \forall n \geq n_0. 0 \leq cf(n) \leq g(n)\}.$

Proviamo separatamente le due inclusioni. Se $g(n) \in \Theta(n)$ allora chiaramente $g(n) \in \Omega(f(n))$, grazie all'esistenza di n_0 e c_1 , e similmente $g(n) \in O(f(n))$, grazie all'esistenza di n_0 e c_2 .

Per il contrario, se $g(n) \in \Omega(f(n))$ e $g(n) \in O(f(n))$, per la prima esistono $c_1 > 0$, $n_1 \in \mathbb{N}$ tali che per ogni $n \geq n_1$ vale

$$0 \leq c_1 f(n) \leq g(n)$$

e per la seconda esistono $c_2 > 0$, $n_2 \in \mathbb{N}$ tali che per ogni $n \geq n_2$ vale

$$0 \leq g(n) \leq c_2 f(n)$$

Quindi, detto $n_0 = \max(n_1, n_2)$ per ogni $n \geq n_0$,

$$0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)$$

ovvero $f(n) \in \Theta(f(n))$.

Domanda 29 Scrivere una funzione *complete*(*T*) che dato in input un albero binario verifica se è completo (ovvero ogni nodo interno ha due figli e tutte le foglie hanno la stessa distanza dalla radice).

Soluzione:

```
complete(x) // verifica se l'albero radicato in x e' completo: ritorna
              // l'altezza dell'albero in caso positivo e -1 in caso negativo
if x = nil
    return 0
else
    countl = complete(x.left)
    countr = complete(x.right)

    if (countr == -1) or (countl == -1) or (countl <> countr)
        return -1
    else
        return countl+1
```

Per quanto riguarda la complessità si osservi che $T(n) = c + T(k) + T(n-k-1)$ per un'opportuna costante c e quindi, la complessità è $O(n)$.

Domanda 30 Scrivere una funzione `diff(T)` che dato in input un albero binario `T` determina la massima differenza di lunghezza tra due cammini che vanno dalla radice ad un sottoalbero vuoto. Ad esempio sull'albero ottenuto inserendo 1, 2 e 3 produce 2, su quello ottenuto inserendo 2, 1, 3 produce 0. Valutarne la complessità.

Soluzione: Il programma si basa su una funzione ricorsiva `diff-rec(x)` che restituisce il la lunghezza del minimo e massimo cammino per il sottoalbero radicato in `x`

```
diff(T)
    min,max = diffRec(T.root)
    return

diff-rec(x)
    if x = nil
        return 0,0
    else
        minl, maxl = diff-rec(x.left)
        minr, maxr = diff-rec(x.right)
        return min(minl,minr)+1, max(maxl,maxr)+1
```

Domanda B (7 punti) Si consideri una tabella hash di dimensione $m = 7$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m - 2)$. Si descriva sinteticamente come avviene l'inserimento degli elementi e si specifichi il risultato dell'inserzione della sequenza di chiavi: 10, 20, 34, 35, 48.

Sarebbe appropriato lavorare con una tabella di dimensione $m = 8$ e le stesse funzioni hash?

Solution

Inserimento sequenza con $m = 7$

Funzioni hash:

- $h_1(k) = k \bmod 7$
- $h_2(k) = 1 + k \bmod 5$ (poiché $m-2 = 5$)

Funzione di probing: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 7$

Tabella iniziale: `T[0..6]` vuota

Sequenza di inserimenti:

k = 10:

- $h_1(10) = 10 \bmod 7 = 3$
- `T[3]` è vuoto $\rightarrow$ inserisco 10 in posizione 3

k = 20:

- $h_1(20) = 20 \bmod 7 = 6$
- `T[6]` è vuoto $\rightarrow$ inserisco 20 in posizione 6

k = 34:

- $h_1(34) = 34 \bmod 7 = 6$
- $T[6]$ è occupato (collisione)
- $h_2(34) = 1 + 34 \bmod 5 = 1 + 4 = 5$
- Probing: $h(34, 1) = (6 + 1 \cdot 5) \bmod 7 = 11 \bmod 7 = 4$
- $T[4]$ è vuoto $\rightarrow$ inserisco 34 in posizione 4

k = 35:

- $h_1(35) = 35 \bmod 7 = 0$
- $T[0]$ è vuoto $\rightarrow$ inserisco 35 in posizione 0

k = 48:

- $h_1(48) = 48 \bmod 7 = 6$
- $T[6]$ è occupato (collisione)
- $h_2(48) = 1 + 48 \bmod 5 = 1 + 3 = 4$
- Probing:
 - $i=1$: $h(48, 1) = (6 + 1 \cdot 4) \bmod 7 = 10 \bmod 7 = 3 \rightarrow$ occupato
 - $i=2$: $h(48, 2) = (6 + 2 \cdot 4) \bmod 7 = 14 \bmod 7 = 0 \rightarrow$ occupato
 - $i=3$: $h(48, 3) = (6 + 3 \cdot 4) \bmod 7 = 18 \bmod 7 = 4 \rightarrow$ occupato
 - $i=4$: $h(48, 4) = (6 + 4 \cdot 4) \bmod 7 = 22 \bmod 7 = 1 \rightarrow$ vuoto
- Inserisco 48 in posizione 1

Tabella finale:

Indice	Chiave
0	35
1	48
2	--
3	10
4	34
5	--
6	20

Appropriatezza di $m = 8$

NO, NON sarebbe appropriato usare $m = 8$ con le stesse funzioni hash.

Motivazione tecnica:

Con $m = 8$:

- $h_1(k) = k \bmod 8$
- $h_2(k) = 1 + k \bmod 6$

Problema critico: Il doppio hashing richiede che $h_2(k)$ e m siano **coprime** ($\text{MCD}(h_2(k), m) = 1$) per garantire che la sequenza di probing esplori tutte le m posizioni della tabella.

Con $m = 8$ (potenza di 2) e $h_2(k) = 1 + k \bmod 6$:

- I possibili valori di $h_2(k)$ sono: $\{1, 2, 3, 4, 5, 6\}$
- I valori **pari** $\{2, 4, 6\}$ hanno $\text{MCD}(h_2(k), 8) = 2 \neq 1$

Conseguenza: Quando $h_2(k)$ è pari, la sequenza di probing visita solo le **4 posizioni pari** o solo le **4 posizioni dispari** (metà della tabella), causando fallimenti prematuri anche con celle disponibili.

Esempio concreto:

Per $k = 20$:

- $h_1(20) = 20 \bmod 8 = 4$
- $h_2(20) = 1 + 20 \bmod 6 = 1 + 2 = 3$ (dispari, funziona)

Per $k = 22$:

- $h_1(22) = 22 \bmod 8 = 6$
- $h_2(22) = 1 + 22 \bmod 6 = 1 + 4 = 5$ (dispari, funziona)

Per $k = 24$:

- $h_1(24) = 24 \bmod 8 = 0$
- $h_2(24) = 1 + 24 \bmod 6 = 1 + 0 = 1$ (dispari, funziona)

Per $k = 26$:

- $h_1(26) = 26 \bmod 8 = 2$
- $h_2(26) = 1 + 26 \bmod 6 = 1 + 2 = 3$ (dispari, funziona)

Per $k = 28$:

- $h_1(28) = 28 \bmod 8 = 4$
- $h_2(28) = 1 + 28 \bmod 6 = 1 + 4 = 5$ (dispari, funziona)

Per $k = 30$:

- $h_1(30) = 30 \bmod 8 = 6$
- $h_2(30) = 1 + 30 \bmod 6 = 1 + 0 = 1$ (dispari, funziona)

Caso problematico - $k = 32$:

- $h_1(32) = 32 \bmod 8 = 0$
- $h_2(32) = 1 + 32 \bmod 6 = 1 + 2 = 3$ (dispari)

Caso problematico - $k = 34$:

- $h_1(34) = 34 \bmod 8 = 2$
- $h_2(34) = 1 + 34 \bmod 6 = 1 + 4 = 5$ (dispari)

Caso critico - $k = 36$:

- $h_1(36) = 36 \bmod 8 = 4$
- $h_2(36) = 1 + 36 \bmod 6 = 1 + 0 = 1$ (dispari)

Correggo l'analisi: con $h_2(k) = 1 + k \bmod 6$, otteniamo sempre valori dispari poiché $k \bmod 6 \in \{0, 1, 2, 3, 4, 5\}$ quindi $1 + k \bmod 6 \in \{1, 2, 3, 4, 5, 6\}$.

Riformulazione corretta del problema:

I valori possibili di $h_2(k)$ sono $\{1, 2, 3, 4, 5, 6\}$.

Quando $h_2(k) \in \{2, 4, 6\}$ (valori pari):

- $\text{MCD}(2, 8) = 2$
- $\text{MCD}(4, 8) = 4$
- $\text{MCD}(6, 8) = 2$

La sequenza di probing **non esplora tutte le 8 posizioni**, ma solo un sottoinsieme, causando fallimenti prematuri.

Conclusione:

$m = 7$ è appropriato perché è primo, garantendo $\text{MCD}(h_2(k), 7) = 1$ per ogni valore di $h_2(k) \in \{1, 2, 3, 4, 5\}$.

$m = 8$ NON è appropriato perché crea situazioni in cui $\text{MCD}(h_2(k), 8) > 1$, impedendo l'esplorazione completa della tabella durante il probing.

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue: Si capisce che si scansione in ordine decrescente di colonna perché quando "i" vale 0, allora "j" vale qualcosa

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j-1) \cdot c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $m = \min\{c(i, j) : 0 \leq i \leq j \leq n-1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di m .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

- (a) Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```

COMPUTE(a,b)
n <- length(a)
m = +infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  m <- MIN(m,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  m <- MIN(m,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j-1] * C[i,j+1]
    m <- MIN(m,C[i,j])
return m

```

(ci prendiamo la lunghezza dell'array e inizializziamo il minimo ad infinito (così, qualsiasi prima quantità minima sarà più piccola)

Siccome devo salvare il minimo, ogni volta si fa il confronto tra il minimo attuale e l'indice di C[i, j] appena salvato.

Ricordandosi che:
- "i" va da i ad (n-1), quindi diventa perché abbiamo già inizializzato, (n-2) la scansione
- "j" va da (n-1) a 0 compresi, quindi dato che abbiamo inizializzato, parte da (n-2)

Siccome nuovamente abbiamo un ciclo in cui "i" va ad (n-2) e in quello di inizializzazione andava ad (n-1), la sommatoria è data da (n-2)(n-1)/2

(b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

La sommatoria interna è costituita da soli 1, in quanto richiede una moltiplicazione tra tre numeri interi, nello specifico (n-2), (n-1), n.

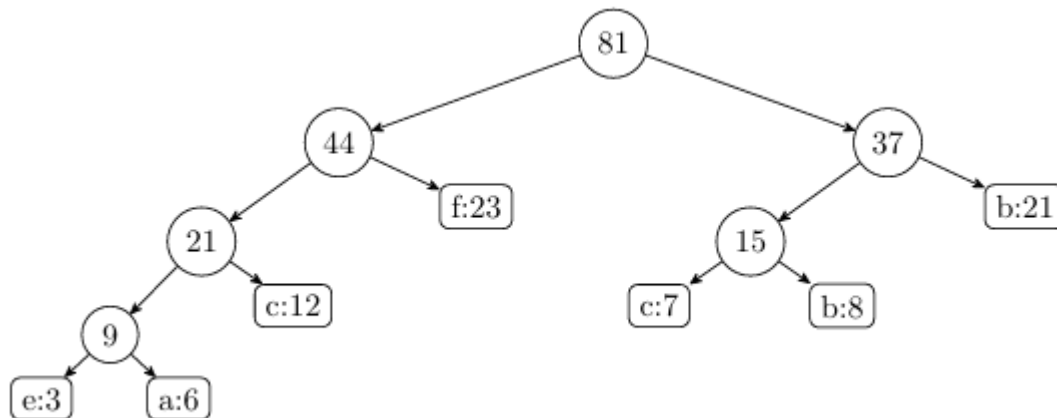
Quanti uno?

Beh, da $j=i+2$ a n ci sono esattamente $n-(i+2)+1=n-i-1$ termini (sostituisco j in i , perché la serie è basata su i e il $+1$ si ha per il fatto che $i=1$). Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$. Non avendo il termine 2 per linearità della sommatoria, non viene moltiplicato con il "fratto 2" di $(n-2)(n-1)$ e quindi il risultato è proprio $(n-2)(n-1)/2$

Domanda 41 Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
6	21	12	8	3	23	8

Spiegare il processo di costruzione del codice.



Esercizio Custom: Funzione con Divide et Impera

Consegna

Esercizio (8 punti) Dato un array $A[1..n]$ di interi distinti, si definisce un **picco** un indice i tale che $A[i-1] < A[i] > A[i+1]$. Per convenzione, si assume $A[0] = A[n+1] = -\infty$, quindi anche il primo e l'ultimo elemento possono essere picchi se soddisfano le condizioni con i loro vicini.

- Scrivere un algoritmo divide-et-impera che trova un picco in A in tempo $O(\log n)$.
- Dimostrare la correttezza dell'algoritmo.

Soluzione

(a) Algoritmo

Idea: Esaminiamo l'elemento centrale. Se è un picco, abbiamo finito. Altrimenti, almeno uno dei due vicini è maggiore: ricorriamo nella metà dell'array che contiene questo vicino maggiore, garantendo che esista un picco in quella metà.

```
Algorithm TROVA_PICCO(A, left, right)
```

```
Input: Array  $A[1..n]$ , indici left e right con  $1 \leq \text{left} \leq \text{right} \leq n$ 
```

```
Output: Un indice  $i$  tale che  $A[i]$  è un picco
```

- if $\text{left} = \text{right}$ then
- return left
-

```

4.  mid ← ⌊(left + right) / 2⌋
5.
6.  // Valori dei vicini (con convenzione A[0] = A[n+1] = -∞)
7.  if mid = 1 then
8.      val_left ← -∞
9.  else
10.     val_left ← A[mid - 1]
11.
12.  if mid = n then
13.     val_right ← -∞
14.  else
15.     val_right ← A[mid + 1]
16.
17.  // Verifica se mid è un picco
18.  if A[mid] > val_left and A[mid] > val_right then
19.      return mid
20.
21.  // Ricorri nella metà promettente
22.  if val_left > A[mid] then
23.      return TROVA_PICCO(A, left, mid - 1)
24.  else
25.      return TROVA_PICCO(A, mid + 1, right)

```

Chiamata iniziale: TROVA_PICCO(A, 1, n)

(b) Correttezza

Teorema: L'algoritmo TROVA_PICCO(A, left, right) restituisce sempre un indice i con $\text{left} \leq i \leq \text{right}$ tale che $A[i]$ è un picco.

Dimostrazione per induzione sulla dimensione del sottoproblema $m = \text{right} - \text{left} + 1$:

Caso base ($m = 1$):

Se $\text{left} = \text{right}$, l'unico elemento nel range è $A[\text{left}]$. Per convenzione, $A[\text{left}-1] = -\infty$ e $A[\text{left}+1] = -\infty$ (o i valori effettivi se esistono), quindi $A[\text{left}]$ è necessariamente un picco. L'algoritmo restituisce left (riga 2). ✓

Passo induttivo:

Assumiamo l'algoritmo corretto per tutti i sottoproblemi di dimensione $< m$. Consideriamo un sottoproblema di dimensione $m \geq 2$.

Calcoliamo $\text{mid} = \lfloor (\text{left} + \text{right}) / 2 \rfloor$.

Caso 1: $A[\text{mid}] > A[\text{mid}-1]$ e $A[\text{mid}] > A[\text{mid}+1]$

Allora mid è un picco per definizione. L'algoritmo restituisce mid (riga 19). ✓

Caso 2: $A[\text{mid}-1] > A[\text{mid}]$

L'algoritmo ricorre su TROVA_PICCO(A, left, mid-1) (riga 23).

Claim: Esiste almeno un picco nel range $[\text{left}, \text{mid}-1]$.

Dimostrazione del claim:

- Consideriamo il cammino da left a $\text{mid}-1$ seguendo sempre gli elementi crescenti
- Partiamo da $A[\text{left}]$: se $A[\text{left}] > A[\text{left}+1]$, allora left è un picco
- Altrimenti, ci spostiamo verso destra finché troviamo un elemento $A[j]$ con $j < \text{mid}$ tale che $A[j] > A[j+1]$
- Tale elemento esiste perché $A[\text{mid}-1] > A[\text{mid}]$, quindi il cammino crescente deve interrompersi
- Quando troviamo $A[j] > A[j+1]$ e sappiamo che $A[j-1] < A[j]$ (dal cammino crescente), allora j è un picco ✓

La dimensione del sottoproblema è $< m$, quindi per ipotesi induttiva l'algoritmo trova correttamente un picco.

Caso 3: $A[\text{mid}+1] > A[\text{mid}]$ (e non vale il Caso 2)

Simmetrico al Caso 2. L'algoritmo ricorre su $\text{TROVA_PICCO}(A, \text{mid}+1, \text{right})$ (riga 25), e per argomento simmetrico esiste almeno un picco nel range $[\text{mid}+1, \text{right}]$. ✓

Per induzione, l'algoritmo è corretto per ogni dimensione del sottoproblema. ■

(c) Complessità

Equazione di ricorrenza:

- **Caso base:** $T(1) = \theta(1)$
- **Caso ricorsivo:** $T(n) = T(n/2) + \theta(1)$

L'algoritmo esegue un numero costante di operazioni (confronti, calcolo di mid) e ricorre su un sottoproblema di dimensione $n/2$.

Applicazione del Teorema Master:

Forma: $T(n) = a \cdot T(n/b) + f(n)$ con $a = 1$, $b = 2$, $f(n) = \theta(1)$

Calcoliamo $n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$

Confronto tra $f(n) = \theta(1)$ e $n^{(\log_b a)} = \theta(1)$:

$f(n) = \theta(n^{(\log_b a)}) \rightarrow$ **Caso 2 del Teorema Master**

Soluzione: $T(n) = \theta(n^{(\log_b a)} \cdot \log n) = \theta(1 \cdot \log n) = \theta(\log n)$

Complessità finale: $T(n) = \Theta(\log n)$

Esercizio 3: Funzione con Albero Binario di Ricerca

Consegna

Esercizio (8 punti) Sia T un albero binario di ricerca (BST) contenente n nodi con chiavi intere distinte. Si vuole progettare un algoritmo che, dati due interi a e b con $a \leq b$, restituisca il numero di nodi in T la cui chiave è compresa nell'intervallo $[a, b]$ (estremi inclusi).

(a) Scrivere un algoritmo ricorsivo efficiente per risolvere il problema.

(b) Determinare la complessità nel caso pessimo dell'algoritmo in funzione di n e h (altezza dell'albero), giustificando la risposta.

Soluzione

(a) Algoritmo

Idea: Sfruttiamo la proprietà di ordinamento del BST per potare rami dell'albero che sicuramente non contengono nodi nell'intervallo $[a, b]$. Se la chiave del nodo corrente è minore di a , tutti i nodi nel sottoalbero sinistro hanno chiavi $< a$, quindi possiamo ignorarlo. Simmetricamente per il sottoalbero destro quando la chiave è maggiore di b .

```
Algorithm CONTA_RANGE( $x, a, b$ )
```

```
Input: Puntatore  $x$  alla radice di un BST, interi  $a$  e  $b$  con  $a \leq b$ 
```

```
Output: Numero di nodi nel sottoalbero radicato in  $x$  con chiave in  $[a, b]$ 
```

```
1. if  $x = \text{NIL}$  then
2.     return 0
3.
4. count  $\leftarrow 0$ 
5.
6. // Se la chiave corrente è nell'intervallo, la contiamo
7. if  $x.\text{key} \geq a$  and  $x.\text{key} \leq b$  then
8.     count  $\leftarrow 1$ 
9.
10. // Ricorriamo sul sottoalbero sinistro se può contenere chiavi  $\geq a$ 
11. if  $x.\text{key} > a$  then
12.     count  $\leftarrow$  count + CONTA_RANGE( $x.\text{left}, a, b$ )
13.
14. // Ricorriamo sul sottoalbero destro se può contenere chiavi  $\leq b$ 
15. if  $x.\text{key} < b$  then
16.     count  $\leftarrow$  count + CONTA_RANGE( $x.\text{right}, a, b$ )
17.
18. return count
```

Chiamata iniziale: CONTA_RANGE($T.\text{root}, a, b$)

Correttezza

Teorema: L'algoritmo `CONTA_RANGE(x, a, b)` restituisce il numero esatto di nodi nel sottoalbero radicato in `x` con chiave nell'intervallo `[a, b]`.

Dimostrazione per induzione strutturale sull'albero:

Caso base (albero vuoto):

Se `x = NIL`, l'albero è vuoto e non contiene nodi. L'algoritmo restituisce 0 (riga 2). ✓

Passo induttivo:

Assumiamo l'algoritmo corretto per tutti i sottoalberi di dimensione minore. Consideriamo un nodo `x` con sottoalberi sinistro e destro.

Sia `S_x` l'insieme dei nodi nel sottoalbero radicato in `x` con chiave in `[a, b]`.

Possiamo partizionare `S_x` in tre sottoinsiemi disgiunti:

1. `S_root = {x}` se $a \leq x.key \leq b$, altrimenti `S_root = ∅`
2. `S_left` = nodi nel sottoalbero sinistro con chiave in `[a, b]`
3. `S_right` = nodi nel sottoalbero destro con chiave in `[a, b]`

Quindi $|S_x| = |S_{root}| + |S_{left}| + |S_{right}|$.

Analisi delle tre componenti:

1. Conteggio del nodo corrente (righe 7-8):

- Se $a \leq x.key \leq b$, `count` viene incrementato di 1 $\rightarrow |S_{root}| = 1$ ✓
- Altrimenti, `count` rimane 0 $\rightarrow |S_{root}| = 0$ ✓

2. Ricorsione sul sottoalbero sinistro (righe 11-12):

- **Proprietà BST:** Tutti i nodi in `x.left` hanno chiave $< x.key$
- **Caso 2a:** Se $x.key > a$, possono esistere nodi in `x.left` con chiave $\geq a$
L'algoritmo ricorre su `x.left`. Per ipotesi induttiva, restituisce correttamente `|S_left|` ✓
- **Caso 2b:** Se $x.key \leq a$, tutti i nodi in `x.left` hanno chiave $< x.key \leq a$, quindi $< a$
Nessun nodo nel sottoalbero sinistro può essere in `[a, b]`, quindi `|S_left| = 0`
L'algoritmo non ricorre (la condizione riga 11 è falsa) e non aggiunge nulla $\rightarrow$ corretto ✓

3. Ricorsione sul sottoalbero destro (righe 15-16):

- **Proprietà BST:** Tutti i nodi in `x.right` hanno chiave $> x.key$
- **Caso 3a:** Se $x.key < b$, possono esistere nodi in `x.right` con chiave $\leq b$
L'algoritmo ricorre su `x.right`. Per ipotesi induttiva, restituisce correttamente `|S_right|` ✓

- **Caso 3b:** Se $x.key \geq b$, tutti i nodi in $x.right$ hanno chiave $> x.key \geq b$, quindi $> b$
Nessun nodo nel sottoalbero destro può essere in $[a, b]$, quindi $|S_{right}| = 0$
L'algoritmo non ricorre (la condizione riga 15 è falsa) e non aggiunge nulla $\rightarrow$ corretto $\checkmark$

Quindi $count = |S_{root}| + |S_{left}| + |S_{right}| = |S_x| \checkmark$

Per induzione strutturale, l'algoritmo è corretto. ■

(b) Complessità

Analisi caso pessimo:

L'algoritmo visita un nodo x solo se esiste un cammino dalla radice a x composto da nodi per cui le condizioni di ricorsione (righe 11 e 15) sono vere.

Caso pessimo: Albero completamente sbilanciato (lista lineare) con altezza $h = n-1$, e l'intervallo $[a, b]$ comprende tutte le chiavi dell'albero.

In questo caso:

- Ogni nodo viene visitato esattamente una volta
- Per ogni nodo, l'algoritmo esegue $O(1)$ operazioni (confronti e somme)
- Numero totale di nodi visitati: n

Complessità in termini di n : $T(n) = O(n)$

Complessità in termini di h :

Nel caso di un BST bilanciato con $h = O(\log n)$:

- Albero con tutte le chiavi in $[a, b]$: visitiamo tutti gli n nodi $\rightarrow T(n) = O(n)$
- Intervallo vuoto o molto piccolo: visitiamo $O(h)$ nodi lungo i cammini di potatura $\rightarrow T(n) = O(h)$

Bound generale più preciso:

Sia k il numero di nodi con chiave in $[a, b]$. L'algoritmo visita:

- Tutti i k nodi dell'output
- Al massimo $O(h)$ nodi aggiuntivi per i cammini di potatura

Complessità caso pessimo: $T(n, h) = O(\min(n, k + h))$

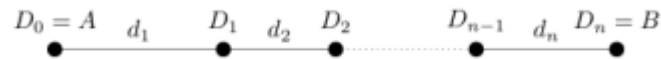
Dove:

- n è il numero totale di nodi nell'albero
- h è l'altezza dell'albero
- k è il numero di nodi nell'intervallo $[a, b]$

In pratica:

- **BST bilanciato:** $h = O(\log n) \rightarrow T = O(k + \log n)$
- **BST sbilanciato:** $h = O(n) \rightarrow T = O(n)$ nel caso peggiore

Esercizio 33 Si supponga di voler viaggiare dalla città A alla città B con un'auto che ha un'autonomia pari a d km. Lungo il percorso si trovano $n - 1$ distributori $D_1, \dots, D_{n-1}$, a distanze di $d_1, \dots, d_n$ km ($d_i \leq d$) come indicato in figura



L'auto ha inizialmente il serbatoio pieno e l'obiettivo è quello di percorrere il viaggio da A a B , minimizzando il numero di soste ai distributori per il rifornimento.

- Introdurre la nozione di soluzione per il problema e di costo della una soluzione. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.
- Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy `stop(d,n)` che dato in input l'array delle distanze `d[1..n]` restituisce una soluzione ottima.
- Valutare la complessità dell'algoritmo.

Soluzione. In generale, la specifica del problema consiste in una sequenza di soste possibili $D_0(=A), D_1, \dots, D_n(=B)$. Per una coppia di soste D_i, D_j con $i \leq j$ poniamo $d_{i,j} = \sum_{h=i+1}^j d_h$. Quindi una soluzione del problema è una sottosequenza di soste che porta dal punto iniziale al punto finale, senza percorrere tratti di lunghezza maggiore di d ovvero:

$$S = D_{i_0} \dots D_{i_k}$$

(con $i_0 < i_1 < \dots < i_k$) tali che $D_{i_0} = A$ e $D_{i_k} = B$, per ogni $j \in \{0, \dots, k-1\}$ vale $d_{i_j, i_{j+1}} \leq d$. Il costo $c(S)$ è il numero $k-1$ di soste.

Sottostruttura ottima. Sia $S = D_{i_0} \dots D_{i_k}$ una soluzione ottima per il problema $D_0, D_1, \dots, D_n$. Se consideriamo il sottoproblema di andare da D_{i_1} a D_n , ovvero $D_{i_1}, D_{i_1+1} \dots D_n$, allora $S_1 = D_{i_1}, \dots, D_{i_k}$ è una soluzione ottima. Infatti, è chiaramente una soluzione. Inoltre, se ci fosse una soluzione migliore S'_1 , con un numero inferiore di soste per il sottoproblema, ovvero $c(S'_1) < c(S_1)$, aggiungendo la sosta D_{i_0} otterremmo una soluzione S' migliore di S per il problema originale dato che $c(S') = c(S'_1) + 1 < c(S_1) + 1 = c(S)$.
 Significa che, avendo scelto bene prima, aggiungendo altre soste, la scelta rimane minima.

Scelta greedy. Per il problema $D_0, D_1, \dots, D_n$, si fissa necessariamente $i_0 = 0$, e la scelta greedy consiste nel raggiungere la sosta più lontana a distanza minore o uguale di d , ovvero definire $i_1 = \max\{j \mid d_{0,j} \leq d\}$.

Data una soluzione ottima per il problema $S = D_{j_0} \dots D_{j_k}$, certamente $j_0 = 0 = i_0$ e $j_1 \leq i_1$. Quindi è immediato verificare che anche $S' = D_{j_0} D_{i_1} D_{j_2} \dots D_{j_k}$ è una soluzione per il problema $D_0, D_1, \dots, D_n$, ed è ottima, dato che $c(S') = c(S)$. (Si noti che, più precisamente, in prima istanza si potrebbe pensare che D_{i_1} potesse essere anche oltre D_{j_2} , ma questo darebbe una soluzione migliore di quella ottima, portando ad un assurdo).

Algoritmo. Ne segue l'algoritmo che riceve in la sequenza di distanze nella forma di un array $d[1..n]$ e restituisce un array $S[1..n-1]$ con le soste scelte (la prima e l'ultima sono scelte sempre, non serve indicarlo).

```
stop(d, n)
  dist = d[1]           // distanza percorsa
  for i=2 to n
    if dist + d[i] > d
      S[i-1]=1
      dist=d[i]
    else
      S[i-1] = 0
  return S
```



Per scelta greedy, prendiamo la sosta con distanza $\leq$ a quella attuale, tale da capire caso per caso quale è conviene di più.

L'algoritmo ricalca proprio la selezione delle attività e:
 - prende come distanza la prima, per inizializzazione
 - se la somma tra la prima distanza e quella attuale è $>$ della sequenza delle distanze, allora abbiamo la distanza buona e salviamo la sosta con la distanza ottima (sapendo che ci siamo fermati "prima", quindi $S[i-1]=1$)
 - se invece non ci siamo fermati prima, avremo $S[i-1] = 0$

La complessità è $\Theta(n)$.

Domanda 39 [5] Scrivere una funzione *blackHeight(T)* che dato in input un albero binario di ricerca *T*, i cui nodi *x* hanno, oltre ai campi *x.key*, *x.left* e *x.right*, hanno un campo *x.col* che può essere *B* (per “black”) oppure *R* (per “red”), verifica se per ogni nodo, il cammino da quel nodo a qualsiasi foglia contiene lo stesso numero di nodi neri (altezza nera). In caso negativo, restituisce -1 , altrimenti restituisce l’altezza nera della radice.

Soluzione:

```
blackHeight(x) // calcola l'altezza nera di x
  if x = nil
    return 0
  else
    countl = blackHeight(x.left)
    countr = blackHeight(x.right)

    // le altezze nere sono diverse se ho un sottoalbero con altezze diverse
    // oppure entrambi i sottoalberi sono non vuoti e con altezze nere diverse

    if (countr == -1) or (countl == -1) or
      (x.left <> nil and x.right <> nil and countl <> countr)
      count=-1
    else
      if (x.left == nil)
        count = countr
      else
        count = countl

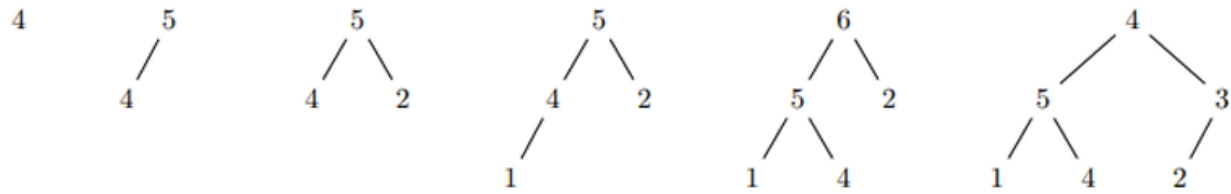
      if x.col == B
        count = count+1

    return count
```

Per quanto riguarda la complessità si osservi che $T(n) = c + T(k) + T(n - k)$ per un’opportuna costante c e quindi, la complessità è $O(n)$.

Domanda B (5 punti) Si dia la definizione della struttura dati max-heap. Si supponga di inserire in un max-heap inizialmente vuoto, in successione, gli elementi 4, 5, 2, 1, 6, 3. Fornire la rappresentazione (ad albero) del max-heap così ottenuto. Si motivi la risposta mostrando la sequenza di max-heap ottenuti.

Soluzione: Per la definizione di max-heap e la procedura di inserzione di un elemento si rimanda al testo. La sequenza di max-heap ottenuta con l’inserimento degli elementi nell’ordine indicato è:



mentre invece la sequenza di min-heap è:

